CPP_STL Project Note

C++ STL and Algorithm Practice Workspace

November 6, 2026

Contents

1 Project Overview **2**

2 Tools and Environment **2**

3 Topics Covered **2**
- 3.1 Input, Output, and Arrays . . . 2
- 3.2 STL Containers . . . 2
- 3.3 Sorting and Custom Comparison . . . 3
- 3.4 Recursion . . . 3

4 Core Algorithm: Custom Interval Array Construction **3**
- 4.1 Algorithm Steps . . . 3
- 4.2 Complexity . . . 4

5 Important Source Files **4**

6 Current Project Structure **4**

7 Code Quality Notes **4**

8 Conclusion **5**

1 Project Overview

The `CPP_STL` project is a hands-on C++ learning workspace. Its goal is to practise the Standard Template Library (STL), fundamental algorithms, recursion, sorting, and competitive-programming style problem solving. The repository currently contains 40 C++ source files. These files are mostly small, independent exercises rather than one combined application.

The project is useful for learning how to select a data structure, translate an algorithm into C++, read input efficiently, and reason about time complexity. jhxjn hjnhbkjwens ,ikdem,a

2 Tools and Environment

- Language: C++
- Typical compiler: `g++` with the C++17 standard
- Editor: VS Code or VSCodium
- Documentation build: LaTeX Workshop with `latexmk`
- Main library: the C++ Standard Template Library (STL)

For a normal C++ source file, a useful compile command is:

```
g++ -std=c++17 -Wall -Wextra -Wpedantic filename.cpp -o program
./program
```

3 Topics Covered

3.1 Input, Output, and Arrays

Several files demonstrate basic input/output, arrays, and vectors. Examples include `first.cpp`, `djsn.cpp`, `dguvhj.cpp`, `jcvnbk.cpp`, and `stl.cpp`. The key ideas are reading values with `cin`, printing with `cout`, traversing elements with loops, and using `vector` when the size should be managed safely at runtime.

3.2 STL Containers

The project explores common STL containers and their intended use:

- `vector`: dynamic contiguous storage and indexed access.
- `queue`: first-in, first-out processing.
- `stack`: last-in, first-out processing.
- `set` and `multiset`: ordered unique or repeated values.
- `map`: ordered key–value storage.

- `priority_queue`: repeated access to the highest-priority item.

Files such as `sort_a2z.cpp`, `pblm_queue.cpp`, `priotrity_quque.cpp`, `custom_PQ.cpp`, and `custom_PQ_1.cpp` focus on container behaviour and priority queues.

3.3 Sorting and Custom Comparison

Sorting is a recurring theme. The standard form is:

```
sort(values.begin(), values.end());
```

For a special order, C++ accepts a comparator. A comparator determines which element has higher priority or should appear first. This is especially important for `priority_queue`, where the ordering rules can be different from a normal sorted array.

3.4 Recursion

The files `recursion.cpp`, `power.cpp`, `power_extenc.cpp`, and `hjghx.cpp` practise recursive thinking. Typical examples include factorial, exponentiation, and checking whether a target value can be formed from a collection of coin values. Every recursive algorithm needs a correct base case and a step that reduces the problem size.

4 Core Algorithm: Custom Interval Array Construction

The most complete algorithmic implementation is `d_custom_array.cpp`. It constructs an array by repeatedly selecting the longest still-empty interval, placing the next value at its middle, and splitting that interval into two smaller intervals.

4.1 Algorithm Steps

1. Start with the interval `[1, n]`.
2. Store active intervals in a `priority_queue`.
3. Give higher priority to a longer interval.
4. If two intervals have equal length, choose the leftmost one.
5. Place the next number at the middle position. For an even-length interval, the implementation uses the left of the two middle cells.
6. Push the non-empty left and right subintervals back into the queue.
7. Continue until every array position has received a value.

The comparator captures the selection rule:

```
if (len1 != len2) return len1 < len2;
return x.first > y.first;
```

This makes a larger interval appear at the top of the priority queue. If the lengths match, the interval with the smaller left endpoint is selected first.

4.2 Complexity

There are n placements. Each interval insertion or removal from the priority queue costs $O(\log n)$. Therefore, the overall complexity is $O(n \log n)$, while the auxiliary storage is $O(n)$.

5 Important Source Files

File	Main learning objective
`d_custom_array.cpp`	Priority queue, comparator, interval splitting
`stl.cpp`	Basic string input and string size
`sort_a2z.cpp`	Sorting, reversing, multiset, priority queue
`custom_PQ.cpp`	Priority queue experiments
`custom_PQ_1.cpp`	Custom ordering with comparators
`pblm_queue.cpp`	Stream processing with a priority queue
`recursion.cpp`	Recursive factorial
`power.cpp`	Iterative exponentiation
`power_extenc.cpp`	Recursive exponentiation
`hjghx.cpp`	Recursive target/coin search

Other practice files include `fibonbaci.cpp`, `follw.cpp`, `probem1.cpp`, `problem2.cpp`, `problem3.cpp`, `problem4.cpp`, `problem5.cpp`, `problem_3.cpp`, `problen4.cpp`, `tgt.cpp`, and several short experimental files.

6 Current Project Structure

```
CPP_STL/
|-- d_custom_array.cpp       # interval-priority algorithm
|-- custom_PQ.cpp            # priority queue practice
|-- custom_PQ_1.cpp          # custom comparator practice
|-- sort_a2z.cpp             # sorting and STL experiments
|-- recursion.cpp            # recursion example
|-- power.cpp                # iterative power
|-- power_extenc.cpp         # recursive power
|-- pblm_queue.cpp           # priority queue problem practice
|-- latex_sample.tex         # minimal LaTeX example
`-- CPP_STL_Project_Note.tex # this report
```

7 Code Quality Notes

This is an educational workspace, so some files are incomplete drafts or experiments. Before submitting a solution or using a file as a final program, the following checks are recommended:

- Keep one `main()` function in each compilable source file.
- Prefer `vector<int>` over variable-length arrays in standard C++.

- Compile with warnings enabled and fix every warning.
- Verify the exact input format, especially whether the first number is a test-case count.
- Test boundary cases such as $n = 1$, empty input where applicable, repeated values, and even-length intervals.

8 Conclusion

The CPP_STL project provides a practical foundation for C++ programming and competitive programming. Its strongest learning themes are STL containers, priority queues, comparators, sorting, recursion, and interval-based problem solving. The next natural step is to organise finished solutions into named folders, keep one problem statement with each solution, and add test cases for each completed program.